KU LEUVEN

Advanced Computer Architecture

Lab Report 3

Bert Pyck
Hugo Charmant

ELICT: ELECFS
6 maart 2025
Master in Engineering Technology
Academic year 2024-2025

1.1 Task description

The goal of this report is to implement an exercise consisting of two parts:

- Part 1: Warps
- Part 2: Thread divergence

We will then present our findings, provide a comprehensive conclusion, and plot the results.

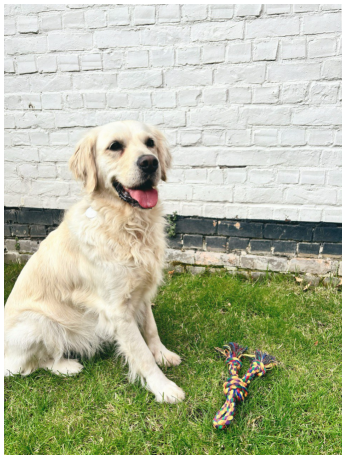
1.2 Introduction

We started the lab session by learning about warps, striding, and divergence. First, we learned what a warp is and its function. We discovered that the reason for the 32-bit limitation is due to hardware constraints.

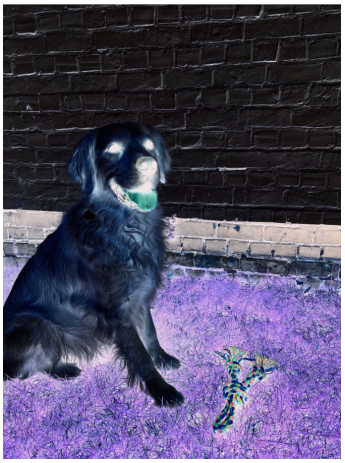
Next, we explored thread divergence within a warp and learned that it must be avoided at all costs. Thread divergence takes place when different operations are happening at the same time in the same warp. This leads to a sequential behaviour, where the first operation is done, and only then is the second operation is performed. Effectively losing a part of the parallel behaviour. To prevent divergence, the same operation should be performed on elements that are a multiple of 32.

We were then introduced to striding, which is a computing method that is necessary when there are more elements than threads. For example, if we have an array of size 64 and we only have 16 threads, we need to iteratively perform our operation on the array, 16 elements at a time. We also had a brief introduction to these concepts in the first lab.

To demonstrate our understanding, we completed an exercise consisting of two parts. The first part involves using warps to implement a kernel that inverts the colours of an image and answering related questions. The second part focuses on thread divergence, where we will modify the inversion kernel to address divergence issues. The input image and the achieved outputs can be found in figure 1.2



Figuur 1.1: Normal picture



Figuur 1.2: Invert

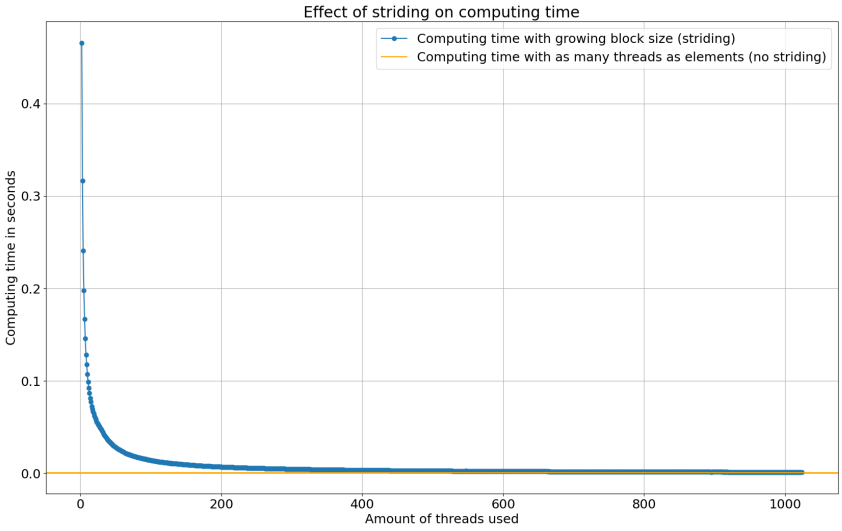


Figuur 1.3: Invert and blur

1.3 Results and conclusion

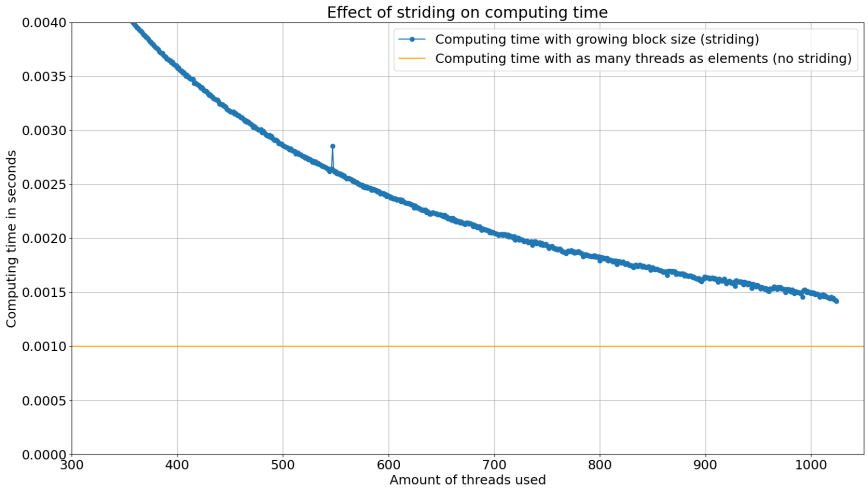
1.3.1 Color inversion - Striding

In order to measure the effect of striding, the inversion operation of each of the color channels of the original picture were performed using different setups.
First, we invert the image with as many threads as elements. This means that we created a grid consisting of as many blocks of 1024 threads as needed. Knowing the dimensions of the picture, we deduce the number of elements (bytes) as Height x Width x 3 (color channels). The corresponding computing time is our reference value.
Next, we showed the effect of striding by only using one block, consisting of a growing size of threads, from 1 to 1024 threads. To measure the performance of these calculations, we measured the computing time at each block size. The results from these measurements can be found in figure 1.5. In order to account for other processes running on the computer, we performed an average of the computing time over multiple iterations.



Figuur 1.4: Computing time VS amount of threads

We can clearly see the effect of striding in this graph. The orange line represents the 'reference time' when we used as many threads as elements. The close up in figure 1.5 shows that, even when using 1024 thread blocks, the striding is still slower than using a bigger amount of threads. Although slower, we note that the difference is quite minimal. Considering the difference in amount of threads used (3686400 threads VS 1024 threads), we conclude that depending on the situation, striding can offer a good balance between performance and resources.



Figuur 1.5: Close up of the results

1.3.2 Color inversion and blurring - Thread Divergence

The objective is again to perform color inversion and blurring using CUDA. Now, we aim to analyze the importance and impact of thread divergence, especially on computation time. The calculations are performed while comparing two different memory layouts: RGBRGBRGB... and RRRGGGBBB.... All elements in the red channel undergo a 1D Gaussian blur, while the green and blue channels undergo a standard inversion. The results are shown in Figure 1.2.

CUDA threads operate in warps (32 threads). When execution paths differ within a warp, it leads to thread divergence, reducing efficiency and performance. Since the RGB format interleaves color channels, processing only red pixels results in conditional checks, causing thread divergence. This occurs in the *invertImageDivergenceRGBRGBRGB*-kernel. There is a way to bypass this occurrence. We need to filter/sort the data into a separate memory layout and then process each color sequentially. This reduces divergence. This approach is implemented in the second kernel *invertImageDivergenceRRRGGGBBB*, with the help of the *sortRGBtoRRRGGGBBB*-function before the kernel and the *sortRRRGGGBBBtoRGB*-function after the kernel. These functions ensure the correct memory layout for computations before the kernel and restore the original layout for displaying the image after the kernel. The code performs these calculations 1000 times to cancel out the effect of other processes on the computing time. In Figure 1.6, we observe that sorting the elements can lead to results twice as fast, demonstrating the significant impact of thread divergence on calculation times.

```
Start RGBRGBRGB
RGBRGBRGB: 0.00876816 sec
Start RRRGGGBBB
RRRGGGBBB: 0.00306354 sec
 *  Terminal will be reused by tasks, press any key to close it.
```

Figuur 1.6: Timing results of with and without sorting

2 CODE

```
#include <iostream>
#include <cstdint>
#include <cuda_runtime.h>
#include <chrono>

#define M 960 // Image width
#define N 1280 // Image height
#define C 3 // Colors (RGB)
#define OFFSET 16 // Header length

double resultFullThreads = 0;
double resultStriding = 0;

int threadsPerBlock;
int blocksPerGrid;

int totalElements = M * N * C;

uint8_t* get_image_array() {
    FILE *imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }

    uint8_t* image_array = (uint8_t*)malloc(M * N * C + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C + OFFSET, imageFile);
    fclose(imageFile);

    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

```

```
__global__ void invertImageStriding(const uint8_t *input, uint8_t *output, int totalElements) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // This for loop ensures striding and total processing of all the elements
    for (int i = idx; i < totalElements; i += blockDim.x * gridDim.x) {
        output[i] = 255 - input[i];
    }
}

int main() {
    uint8_t* image_array = get_image_array();
    uint8_t* invert_array = (uint8_t*)malloc(M * N * C);
    uint8_t *d_input, *d_output;

    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);

    cudaMalloc((void**)&d_input, M * N * C * sizeof(uint8_t));
    cudaMalloc((void**)&d_output, M * N * C * sizeof(uint8_t));

    cudaMemcpy(d_input, image_array, M * N * C * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    for (int j=0;j<100;j++){
        cudaEventRecord(start_cuda);

        // First iteration, with as many threads that there are elements
        threadsPerBlock = 1024;
        blocksPerGrid = (totalElements + threadsPerBlock - 1) / threadsPerBlock;

        // Kernel invocation
        invertImageStriding<<<blocksPerGrid, threadsPerBlock>>>(d_input, d_output,
             totalElements);

        cudaDeviceSynchronize();
        cudaEventRecord(stop_cuda);
        cudaEventSynchronize(stop_cuda);

        cudaMemcpy(invert_array, d_output, M * N * C * sizeof(uint8_t),
            cudaMemcpyDeviceToHost);
        float ms;
        cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
        resultFullThreads += ms/1000;
    }

    std::cout << resultFullThreads/100 << "␣seconds␣for␣as␣many␣threads␣as␣
        elements" << std::endl;


    // Now we will see the effect of striding, using one block and a growing
        number of threads.


    blocksPerGrid = 1;
```

```
95
96      for (int i = 1; i <= 1024; i++){
97          for (int j = 0; j < 5; j++){
98              threadsPerBlock = i;
99              cudaEvent_t start_cuda, stop_cuda;
100             cudaEventCreate(&start_cuda);
101             cudaEventCreate(&stop_cuda);
102
103             cudaMemcpy(d_input, image_array, M * N * C * sizeof(uint8_t),
                    cudaMemcpyHostToDevice);
104
105             cudaEventRecord(start_cuda);
106
107             invertImageStriding<<<blocksPerGrid, threadsPerBlock>>>(d_input,
                    d_output, totalElements);
108             cudaDeviceSynchronize();
109
110             cudaEventRecord(stop_cuda);
111             cudaEventSynchronize(stop_cuda);
112
113             cudaMemcpy(invert_array, d_output, M * N * C * sizeof(uint8_t),
                    cudaMemcpyDeviceToHost);
114
115             float ms;
116             cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
117             resultStriding += ms/1000;
118         }
119         std::cout << threadsPerBlock << "," << resultStriding/5 << std::endl;
120         resultStriding = 0;
121     }
122
123     save_image_array(invert_array);
124
125     cudaFree(d_input);
126     cudaFree(d_output);
127     free(image_array - OFFSET);
128     free(invert_array);
129     return 0;
130 }
```

Listing 2.1: Striding

```
1  #include <iostream>
2  #include <cstdint>
3  #include <cuda_runtime.h>
4  #include <chrono>
5
6  #define M 960 // Image width
7  #define N 1280 // Image height
8  #define C 3 // Colors (RGB)
9  #define OFFSET 16 // Header length
10 #define BLOCK_SIZE 1024 // Threads per block
11 #define AVERAGE 1000
12
13 int totalElements = M * N * C;
14 double RGBRGBRGBtime = 0;
```

```
15 double RRRGGGBBBtime = 0;
16
17 uint8_t* get_image_array() {
18     FILE *imageFile = fopen("./anna.ppm", "rb");
19     if (imageFile == NULL) {
20         perror("ERROR: Cannot open input file");
21         exit(EXIT_FAILURE);
22     }
23
24     uint8_t* image_array = (uint8_t*)malloc(M * N * C + OFFSET);
25     fread(image_array, sizeof(uint8_t), M * N * C + OFFSET, imageFile);
26     fclose(imageFile);
27
28     return image_array + OFFSET;
29 }
30
31 void save_image_array(uint8_t* image_array) {
32     FILE *imageFile = fopen("./output_image.ppm", "wb");
33     if (imageFile == NULL) {
34         perror("ERROR: Cannot open output file");
35         exit(EXIT_FAILURE);
36     }
37
38     fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
39     fwrite(image_array, 1, M * N * C, imageFile);
40     fclose(imageFile);
41 }
42
43 void sortRGBtoRRRGGGBBB(uint8_t* input, uint8_t* output) {
44     int totalPixels = M * N; // Total number of pixels (not bytes)
45
46     int redIndex = 0;
47     int greenIndex = totalPixels;
48     int blueIndex = 2 * totalPixels;
49
50     for (int i = 0; i < totalPixels; i++) {
51         output[redIndex++] = input[i * 3]; // Copy Red
52         output[greenIndex++] = input[i * 3 + 1]; // Copy Green
53         output[blueIndex++] = input[i * 3 + 2]; // Copy Blue
54     }
55 }
56
57 void sortRRRGGGBBBtoRGB(uint8_t* input, uint8_t* output) {
58     int totalPixels = M * N; // Total number of pixels
59
60     int redIndex = 0;
61     int greenIndex = totalPixels;
62     int blueIndex = 2 * totalPixels;
63
64     for (int i = 0; i < totalPixels; i++) {
65         output[i * 3] = input[redIndex++]; // Copy Red
66         output[i * 3 + 1] = input[greenIndex++]; // Copy Green
67         output[i * 3 + 2] = input[blueIndex++]; // Copy Blue
68     }
69 }
70
```

```
71 __global__ void invertImageDivergenceRGBRGBRGB(const uint8_t *input, uint8_t *
       output, int totalElements) {
72     int idx = blockDim.x * blockIdx.x + threadIdx.x;
73     for (int i = idx; i < totalElements; i += blockDim.x * gridDim.x) {
74         if (i % 3 == 0 && i > 2 && i < totalElements - 2) {
75             // For red channel
76             output[i] = 0.1 * input[i-2] + 0.25 * input[i-1] + 0.5 * input[i] +
                   0.25 * input[i+1] + 0.1 * input[i+2];
77         } else {
78             // For green and blue
79             output[i] = 255 - input[i];
80         }
81     }
82 }
83
84 __global__ void invertImageDivergenceRRRGGGBBB(const uint8_t *input, uint8_t *
       output, int totalElements) {
85     int idx = blockIdx.x * blockDim.x + threadIdx.x;
86     int R_total = M * N; // Total number of red pixels (first 1/3 of the data)
87
88     for (int i = idx; i < totalElements; i += blockDim.x * gridDim.x) {
89         if (i < R_total) {
90             // Red channel - Gaussian blur
91             int prev2 = (i >= 2) ? input[i - 2] : input[i]; // Handle edge case
92             int prev1 = (i >= 1) ? input[i - 1] : input[i];
93             int next1 = (i < R_total - 1) ? input[i + 1] : input[i];
94             int next2 = (i < R_total - 2) ? input[i + 2] : input[i];
95
96             output[i] = 0.1 * prev2 + 0.25 * prev1 + 0.5 * input[i] + 0.25 * next1
                    + 0.1 * next2;
97         } else {
98             // Green and Blue channels - Simple inversion
99             output[i] = 255 - input[i];
100        }
101    }
102 }
103
104 int main() {
105
106     std::cout<<"Start␣RGBRGBRGB"<< std::endl;
107     for(int i = 0; i < AVERAGE; i++){
108         cudaEvent_t start_cuda, stop_cuda;
109         cudaEventCreate(&start_cuda);
110         cudaEventCreate(&stop_cuda);
111
112         //std::cout << "RGBRGBRGB"<<std::endl;
113         uint8_t* image_array = get_image_array();
114         uint8_t* invert_array = (uint8_t*)malloc(M * N * C);
115
116         uint8_t *d_input, *d_output;
117         cudaMalloc((void**)&d_input, M * N * C * sizeof(uint8_t));
118         cudaMalloc((void**)&d_output, M * N * C * sizeof(uint8_t));
119
120         cudaMemcpy(d_input, image_array, M * N * C * sizeof(uint8_t),
               cudaMemcpyHostToDevice);
121
```

```
122         int gridSize = (totalElements + BLOCK_SIZE - 1) / BLOCK_SIZE;
123 
124         cudaEventRecord(start_cuda);
125 
126         invertImageDivergenceRGBRGBRGB<<<gridSize, BLOCK_SIZE>>>(d_input, d_output
                , totalElements);
127         cudaDeviceSynchronize();
128 
129         cudaEventRecord(stop_cuda);
130         cudaEventSynchronize(stop_cuda);
131 
132         cudaMemcpy(invert_array, d_output, M * N * C * sizeof(uint8_t),
                cudaMemcpyDeviceToHost);
133 
134         // Striding have to implement
135 
136         save_image_array(invert_array);
137 
138         cudaFree(d_input);
139         cudaFree(d_output);
140         free(image_array - OFFSET);
141         free(invert_array);
142 
143         float ms;
144         cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
145         RGBRGBRGBtime += ms/1000;
146     }
147     std::cout<<"RGBRGBRGB:␣" << (RGBRGBRGBtime/AVERAGE) << "␣sec" << std::endl;
148 
149 
150 
151     std::cout<<"Start␣RRRGGGBBB"<< std::endl;
152     for(int i = 0; i < AVERAGE; i++){
153         cudaEvent_t start_cuda, stop_cuda;
154         cudaEventCreate(&start_cuda);
155         cudaEventCreate(&stop_cuda);
156 
157         //std::cout << "RRRGGGBBB"<<std::endl;
158         uint8_t* image_array = get_image_array();
159         uint8_t* sorted_array = (uint8_t*)malloc(M * N * C); // Create sorted
                array
160         uint8_t* processed_array = (uint8_t*)malloc(M * N * C);
161         uint8_t* invert_array = (uint8_t*)malloc(M * N * C);
162 
163         sortRGBtoRRRGGGBBB(image_array, sorted_array);
164 
165         uint8_t *d_input, *d_output;
166         cudaMalloc((void**)&d_input, M * N * C * sizeof(uint8_t));
167         cudaMalloc((void**)&d_output, M * N * C * sizeof(uint8_t));
168 
169         cudaMemcpy(d_input, sorted_array, M * N * C * sizeof(uint8_t),
                cudaMemcpyHostToDevice);
170 
171         int gridSize = (totalElements + BLOCK_SIZE - 1) / BLOCK_SIZE;
172 
173         cudaEventRecord(start_cuda);
```

```
174
175         invertImageDivergenceRRRGGGBBB<<<gridSize, BLOCK_SIZE>>>(d_input, d_output
                , totalElements);
176         cudaDeviceSynchronize();
177
178         cudaEventRecord(stop_cuda);
179         cudaEventSynchronize(stop_cuda);
180
181         cudaMemcpy(processed_array, d_output, M * N * C * sizeof(uint8_t),
                cudaMemcpyDeviceToHost);
182
183         sortRRRGGGBBBtoRGB(processed_array, invert_array);
184
185         save_image_array(invert_array);
186
187         cudaFree(d_input);
188         cudaFree(d_output);
189         free(image_array - OFFSET);
190         free(sorted_array);
191         free(processed_array);
192         free(invert_array);
193
194         float ms;
195         cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
196         RRRGGGBBBtime += ms/1000;
197     }
198     std::cout<<"RRRGGGBBB: " << (RRRGGGBBBtime/AVERAGE) << " sec" << std::endl;
199
200
201     return 0;
202 }
```

Listing 2.2: Measuring timing results with and without thread divergence